Human Face Emotion Classification with Multi-Layer Perceptrons

Results and interpretation report

Student: Adel Ahmed Fathi Amer | **ID:** 224100177

Companion file: *emotion_mlp.ipynb* (all cells executed; every number in this report is taken from that run).

Summary

We built a five-class facial emotion classifier (Angry, Fear, Happy, Sad, Surprise) using **only multi-layer perceptrons**. After cleaning the data and testing 13 input representations, 22 architectures, 7 activation functions, many regularisation / optimiser / batch-size / schedule settings, three ways of handling class imbalance, two kinds of augmentation and a 30-trial random search (132 configurations, 243 training runs in total), the best model is an MLP with hidden layers 1024-512 on **HOG + LBP** features. On the held-out test set a single model reaches **60.1 % accuracy / 0.573 macro-F1**, and an ensemble of five such MLPs reaches **62.2 % accuracy / 0.593 macro-F1** (always predicting the majority class gives 32.1 %; a plain MLP on raw pixels gets 47.0 % on validation).

The main findings: (1) the **input representation** matters far more than any other choice (+10 points macro-F1); (2) the dataset contains **51 % exact duplicates**, which would silently inflate any test score if left in; (3) all MLPs memorise the training set (about 100 % training accuracy), so generalisation, not capacity, is the bottleneck - architecture, activation and regularisation changes are mostly within +-1 point; (4) the remaining errors are concentrated on Fear / Angry / Sad, which look alike at 48x48 grayscale.

1. Task, data and protocol

Task and constraint. Classify a face image into one of five emotions using the Kaggle dataset *Human Face Emotions* (samithsachidanandan). Only MLPs are allowed: stacks of fully-connected layers. PyTorch is used only as a fast GPU engine for `nn.Linear` layers (no convolutions, no pre-trained or "advanced" networks); scikit-learn's `MLPClassifier` is used as an independent cross-check. HOG, LBP and PCA are used purely as *input preprocessing* for the MLP.

Data. 59,099 images in five folders (Angry 10,148; Fear 9,732; Happy 18,439; Sad 12,553; "Suprise" [sic] 8,227). 97.7 % are 48x48 grayscale, the remaining ~1,340 are large colour photographs; all images were converted to 48x48 grayscale (Lanczos resizing for the large ones).

Duplicates. Web-collected face datasets often contain repeated images. We hashed every 48x48 image: **30,232 extra copies are exact duplicates** and 49 images appear under two different labels (all copies removed). Only **28,818** of 59,099 images remain (48.8 %). If duplicates were kept, copies of the same photo would land in both training and test sets and the test accuracy would measure memorisation, not generalisation. This is the single most important data-quality step of the study.

Class	Angry	Fear	Happy	Sad	Surprise	Total
Images after cleaning	4,959	4,822	9,255	6,310	3,472	28,818
Share	17.2%	16.7%	32.1%	21.9%	12.0%	100 %

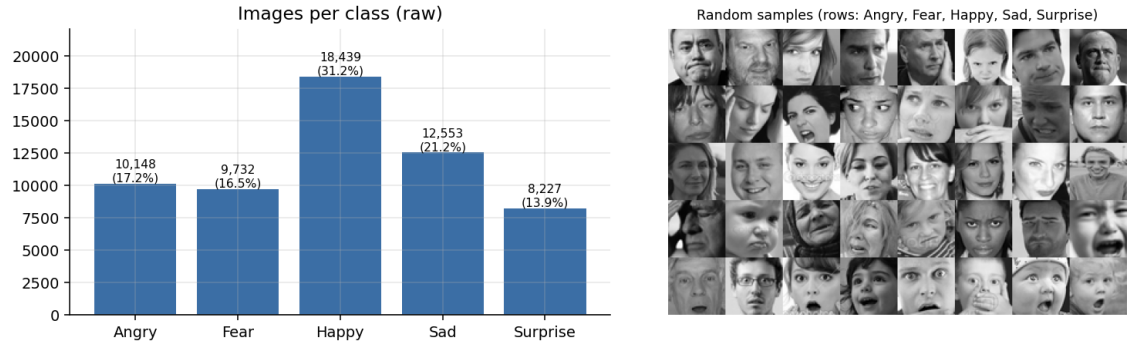


Figure 1. Class distribution of the raw data (left; Happy is over-represented) and random samples (right; rows Angry, Fear, Happy, Sad, Surprise). Faces vary a lot in pose, lighting and style.

Split and protocol. Stratified 70 / 15 / 15 split of the cleaned data: 20,172 train, 4,323 validation, 4,323 test. All statistics learnt from data (means, standard deviations, PCA) are fitted on the training set only. **Every design decision uses the validation set; the test set is used once, for the final model.** Because the classes are imbalanced, the selection metric is **macro-F1** (the mean of the five per-class F1 scores); accuracy is always reported too. Early stopping on validation macro-F1 (patience 10, at most 60 epochs, 80 in the on-the-fly augmentation study) is used in every run, and each configuration is repeated with 2-3 seeds where the differences are small.

2. Models and preprocessing

MLP family. `Linear -> [BatchNorm] -> activation -> [Dropout]` blocks followed by a 5-way linear output layer, trained with cross-entropy. The whole data set lives on the GPU, so a typical run takes 3-10 seconds; the study contains 243 training runs (38 minutes of training in total).

Thirteen input representations were compared with the same baseline MLP (2304 -> 512 -> 256 -> 5, Adam 1e-3, batch 256):

- **Raw pixels:** scaled to [0,1] (*raw01*); global z-score (*global_z*); per-pixel z-score (*pixel_z*); per-image standardisation, i.e. zero mean and unit variance for every image (*img_std*).
- **Contrast enhancement** followed by per-image standardisation: histogram equalisation (*heq_std*) and CLAHE (*clahe_std*); 24x24 area down-sampling (*down24_std*).
- **PCA** of *img_std* to 200 components, with and without whitening (*pca200_white*, *pca200*).
- **Hand-crafted descriptors:** HOG (9 orientations, 6x6 cells, 2x2 blocks; 1,764 values), LBP-histograms on a 6x6 grid (360 values), both concatenated (2,124 values), and HOG reduced by PCA to 256 values.



Figure 2. What the MLP receives: original, per-image standardised, equalised, CLAHE, 24x24 and the HOG gradient visualisation.

3. Experiments and results (validation set)

3.1 Baseline

A plain MLP on raw pixels gives **47.0 % accuracy and 0.433 macro-F1** (3 seeds, std 0.007). Training accuracy rises to about 64 % while the validation loss stops improving after ~10 epochs and then stays flat: the network learns the training pixels but does not generalise. This is the reference for all later improvements.

3.2 Data preprocessing

Representation	Dim.	Val. acc.	Val. macro-F1	Std (2 seeds)	Train acc.
hog+lbp	2124	0.563	**0.532**	0.001	1.00
hog	1764	0.561	**0.527**	0.002	1.00
hog_pca256	256	0.546	**0.510**	0.003	0.86
img_std	2304	0.500	**0.473**	0.003	1.00
heq_std	2304	0.500	**0.469**	0.004	1.00
down24_std	576	0.492	**0.464**	0.000	1.00
global_z	2304	0.490	**0.464**	0.000	0.99
pixel_z	2304	0.492	**0.464**	0.001	0.96
clahe_std	2304	0.484	**0.456**	0.001	1.00
pca200	200	0.476	**0.443**	0.001	0.82
raw01	2304	0.466	**0.429**	0.007	0.57
pca200_white	200	0.458	**0.422**	0.003	0.81
lbp	360	0.437	**0.409**	0.000	1.00

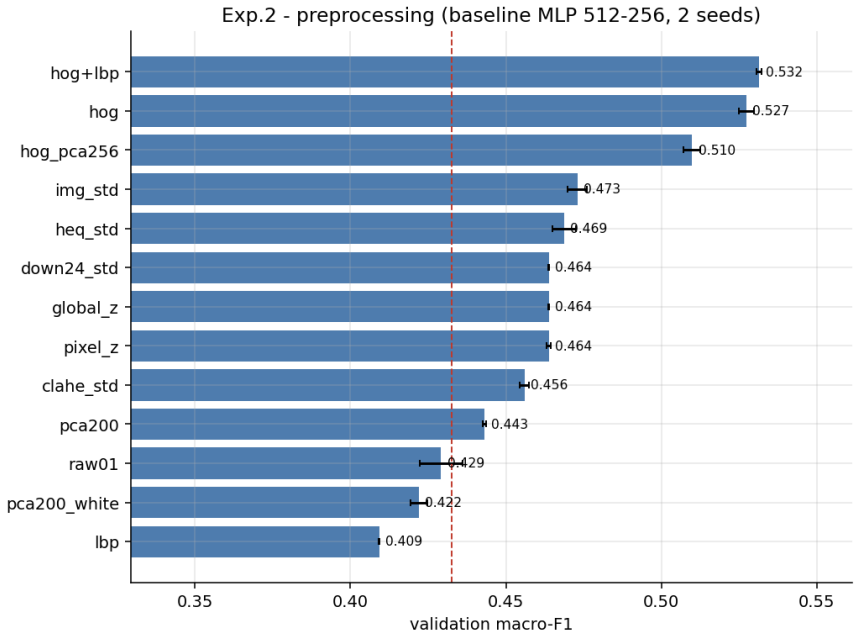


Figure 3. Validation macro-F1 per input representation (dashed line: raw-pixel baseline).

Interpretation. HOG+LBP (0.532) and HOG (0.527) are about 10 points better than the baseline and 6 points better than the best pixel variant. HOG stores local edge orientations in cell histograms, which keeps the information that distinguishes expressions (mouth and eyebrow edges) while being insensitive to brightness and to shifts smaller than a cell; an MLP has no built-in translation invariance, so on pixels it must learn this from ~20k noisy images. Per-image standardisation helps pixel inputs by about 4 points (it removes lighting and contrast differences) whereas histogram equalisation and CLAHE do not (CLAHE amplifies noise in 48x48 images). Down-sampling to 24x24 loses nothing (0.464 vs 0.473 at full size, with 3x fewer parameters), showing that fine pixel detail is not being used. PCA does not help an MLP: whitening is harmful (0.422) because it blows up low-variance, noisy directions. LBP alone is weak (0.409) but adds a little to HOG. Nearly all pipelines reach about 100 % training accuracy, which already signals over-fitting as the main problem. We continue with **HOG+LBP**.

3.3 Architecture (depth, width, shape)

A 4 x 4 grid of constant-width networks and six other shapes were run with two seeds each (validation macro-F1):

Hidden layers \ width	128	256	512	1024
1	0.525	0.540	0.547	0.548
2	0.523	0.529	0.531	0.539
3	0.521	0.523	0.529	0.536
4	0.518	0.524	0.532	0.530

Other shapes: 4096 (wide-shallow) 0.540; 2048-1024-512-256 0.535; 2048-2048 0.534; 1024-512-256 (pyramid) 0.531; 512-128-512 (bottleneck) 0.531; 256-512-1024 (expanding) 0.527.

Interpretation. The best model is a **single wide hidden layer (1 x 1024, 0.548)**; deeper networks are consistently a little worse (with 1024 units: 0.548, 0.539, 0.536, 0.530 for 1-4 layers) and very narrow layers lose 2-3 points. HOG+LBP is already a good representation, so extra non-linear depth mostly adds capacity to memorise the training images rather than useful features. Differences among the top models are about 1 point, close to the seed standard deviation (0.2-0.8 point): the *trend* with depth is reliable, the exact ranking of the top few is not.

3.4 Activation functions

Activation	Val. acc.	Val. macro-F1	Std
relu	0.585	0.548	0.005
leaky_relu	0.582	0.548	0.003
gelu	0.581	0.547	0.005
silu	0.577	0.543	0.003
elu	0.570	0.535	0.000
sigmoid	0.568	0.530	0.003
tanh	0.560	0.524	0.001

Interpretation. ReLU, Leaky-ReLU and GELU are tied (0.547-0.548). SiLU and ELU are slightly lower, and the saturating functions are worst (sigmoid 0.530, tanh 0.525): they squash large HOG responses and train more slowly. ReLU is kept.

3.5 Regularisation

One factor at a time around the current best model (two seeds; a value is adopted only if it beats the base by more than 0.003): dropout 0-0.5, batch-norm, weight decay (AdamW) 1e-4 to 5e-2, label smoothing 0-0.2, Gaussian input noise 0-0.3.

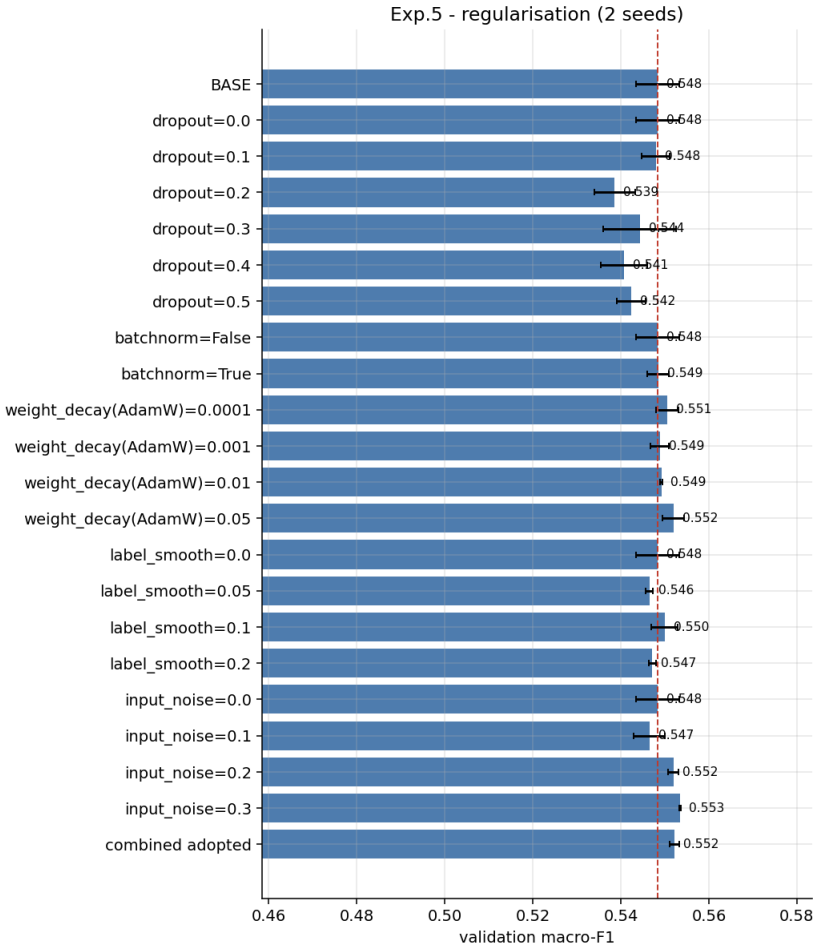


Figure 4. Validation macro-F1 for each regularisation setting (dashed: base 0.548).

Interpretation. Almost nothing moves the score by more than the seed noise: dropout 0.1-0.5 gives 0.539-0.548, batch-norm 0.549, label smoothing 0.547-0.550. Only strong weight decay (0.05: 0.552) and heavy input noise (sigma 0.3: 0.553) are adopted, giving 0.552 combined (+0.4 point). The training accuracy stays at ~100 % throughout: a 2.2 M-parameter network can memorise ~20k images whatever simple regulariser is applied, and a train/validation gap of ~40 points remains.

3.6 Optimiser, learning rate, batch size and schedule

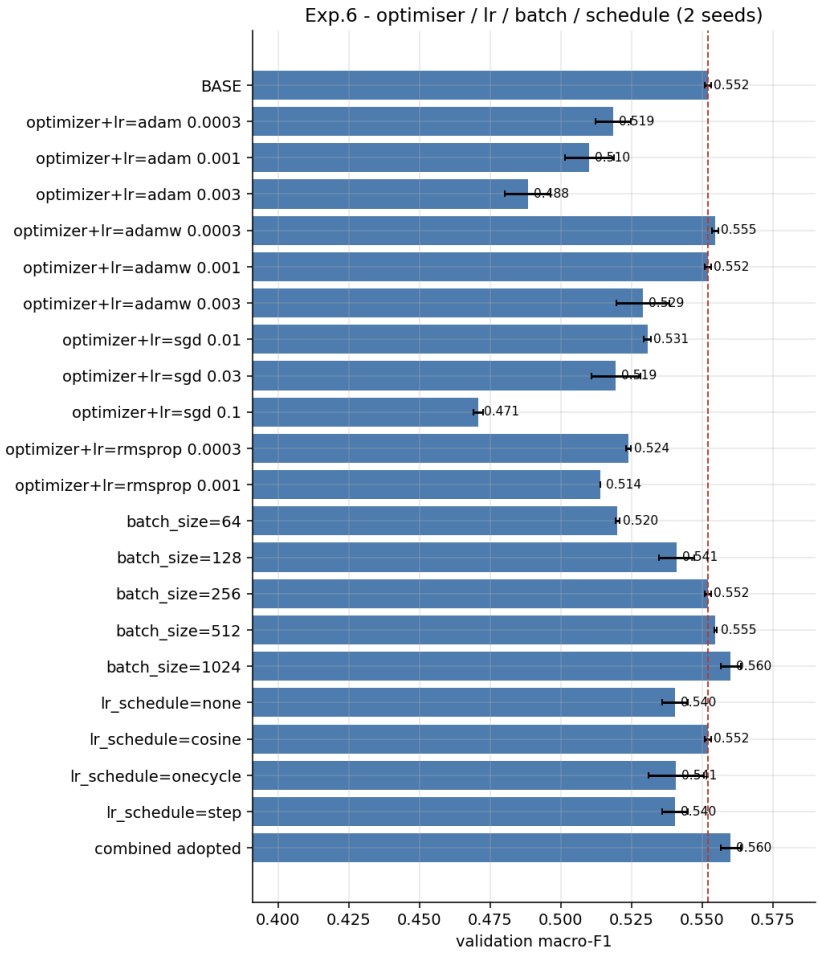


Figure 5. Optimisation settings (2 seeds; dashed: base 0.552). Note the weight-decay confound explained in the text.

Interpretation. Batch size is the most influential setting: 0.520 (64), 0.541 (128), 0.552 (256), 0.555 (512), 0.560 (1024); larger batches give less noisy gradient steps and generalised better here, so 1024 is adopted (+0.8 point). AdamW at 3e-4 and 1e-3 are tied (0.555 / 0.552); 3e-3 is too high (0.529). Cosine decay is ~1 point better than no schedule or one-cycle (the "none" and "step" rows are identical because early stopping ends training before the first step decay). **Caveat:** the base uses AdamW with weight decay 0.05, and for Adam, SGD and RMSprop the same number acts as a coupled L2 penalty, which is much too strong for them (their training accuracy is only 56-75 %). Their lower scores (0.47-0.53) therefore partly reflect this confound and must not be read as proof that AdamW is intrinsically better.

3.7 Class imbalance

Strategy	Angry	Fear	Happy	Sad	Surprise	Macro-F1
none	0.465	0.355	0.821	0.534	0.607	0.558
class-weighted loss	0.470	0.358	0.800	0.533	0.623	0.556
oversampling	0.478	0.352	0.811	0.521	0.608	0.555

Validation recall per class (3 seeds). **Interpretation.** Weighted loss and over-sampling do not change macro-F1; they move a little recall from *Happy* (0.82 -> 0.80-0.81) to Surprise / Angry. *Fear* remains the hardest class (recall ~0.35) under every option, so the difficulty is that classes look alike, not that they are rare. Not adopted.

3.8 Data augmentation

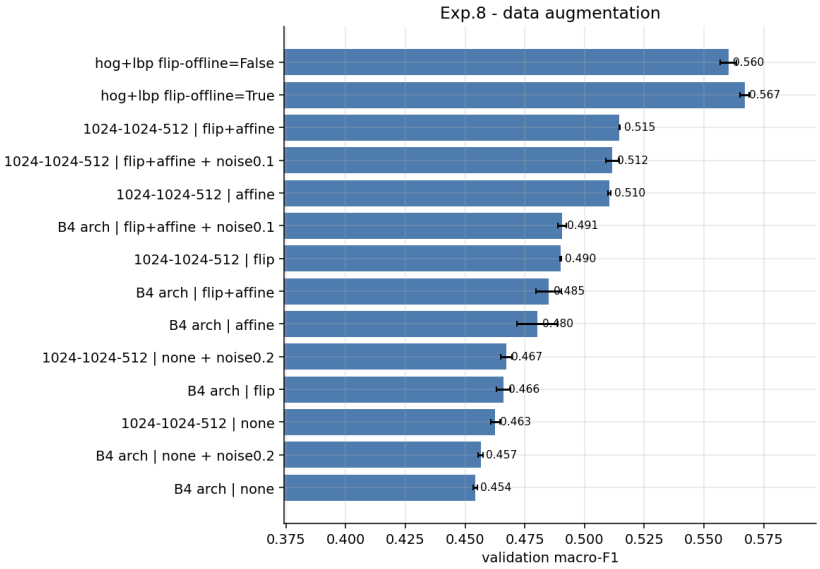


Figure 6. Augmentation. Top two rows: offline horizontal flip on HOG+LBP. Others: on-the-fly flip / affine / noise on raw (per-image standardised) pixels, for a single-layer 1024 MLP (labelled "B4 arch" = the settings adopted up to Section 3.7) and a 1024-1024-512 MLP.

Interpretation. Adding horizontally flipped copies of the training images to the HOG+LBP model gives a small, consistent gain (0.560 -> 0.567); it is adopted. On raw pixels, augmentation is much more valuable: the large MLP improves from 0.463 to 0.515 with flip + small affine transformations, and its training accuracy falls from 100 % to 92 %, i.e. it memorises less. Still, the best augmented pixel model stays below the HOG+LBP model, so the hand-crafted features remain the better input for an MLP.

3.9 Joint random search and cross-check

One-factor-at-a-time tuning ignores interactions, so 30 random configurations (hidden layers, activation, dropout, batch-norm, optimiser, learning rate, weight decay, batch size, label smoothing, class weights, flip augmentation) were also evaluated, with the greedy best model as trial 0 (0.569). The best trials reach 0.570-0.578 with one seed. **All five best trials use flip augmentation and at least two hidden layers, four of five use batch-norm and label smoothing 0.1**: with strong regularisation and augmentation, depth no longer hurts, an interaction that the one-factor sweeps could not show. Re-training the five best with three fresh seeds shrinks their spread to 0.567-0.575 (e.g. trial 09 falls from 0.578 to 0.571), so the top configurations are statistically indistinguishable and the first-seed numbers were optimistic (winner's curse). The tuning has plateaued at ~0.57 validation macro-F1.

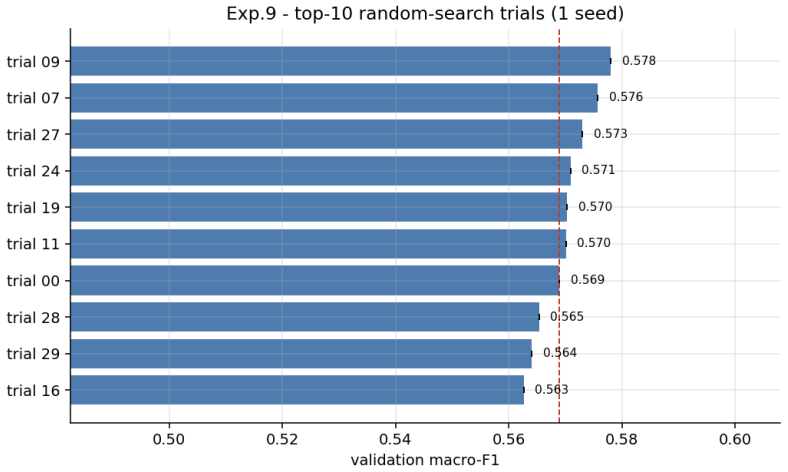


Figure 7. Top-10 random-search trials (single seed; dashed: greedy configuration).

Cross-check with scikit-learn. An independent `MLPClassifier` (512-256, adam, early stopping) reaches 0.558 accuracy / 0.524 macro-F1 on HOG+LBP and 0.448 / 0.407 on whitened PCA features - the same ranking and almost the same values as our PyTorch baseline (0.563 / 0.532), which supports the correctness of our training code and of the conclusions on preprocessing.

3.10 Progress of the tuning

Stage	Val. acc.	Val. macro-F1
1 baseline (raw pixels, 512-256)	0.470	0.433
2 + best preprocessing	0.563	0.532
3 + best architecture	0.585	0.548
4 + best activation	0.585	0.548
5 + regularisation	0.589	0.552
6 + optimiser/lr/batch/sched	0.595	0.560
7 final (re-checked, 3 fresh seeds)	0.607	0.575

Preprocessing (stage 2) delivers +10 points; everything else together adds another ~4-5 points. The last row re-trains the selected configuration with three new seeds (flip augmentation included).

4. Final model and test results

Selected configuration. HOG+LBP features (standardised), flipped training copies added; MLP **1024 -> 512**, GELU, batch-norm, dropout 0.1, label smoothing 0.1, AdamW (lr 2.9e-3, weight decay 2e-3), batch size 256, cosine schedule, early stopping; 2,706,437 parameters. It was chosen as the best of the five top configurations after re-training each with fresh seeds, then trained with five new seeds and evaluated **once** on the test set.

Model	Test accuracy	Test macro-F1
5 single MLPs (mean +- std)	60.1 % +- 0.3	0.573 +- 0.004
Ensemble: average of the 5 MLPs' probabilities	**62.2 %**	**0.593**
Validation (single MLPs, for comparison)	60.5 %	0.572
Majority-class baseline	32.1 %	0.097
Baseline MLP on raw pixels (validation)	47.0 %	0.433

Per-class test metrics of the ensemble:

Class	Precision	Recall	F1	Support
Angry	0.506	0.487	0.496	744
Fear	0.476	0.401	0.435	724
Happy	0.771	0.823	0.796	1388
Sad	0.534	0.589	0.561	947
Surprise	0.712	0.646	0.677	520
macro average	**0.600**	**0.589**	**0.593**	**4323**

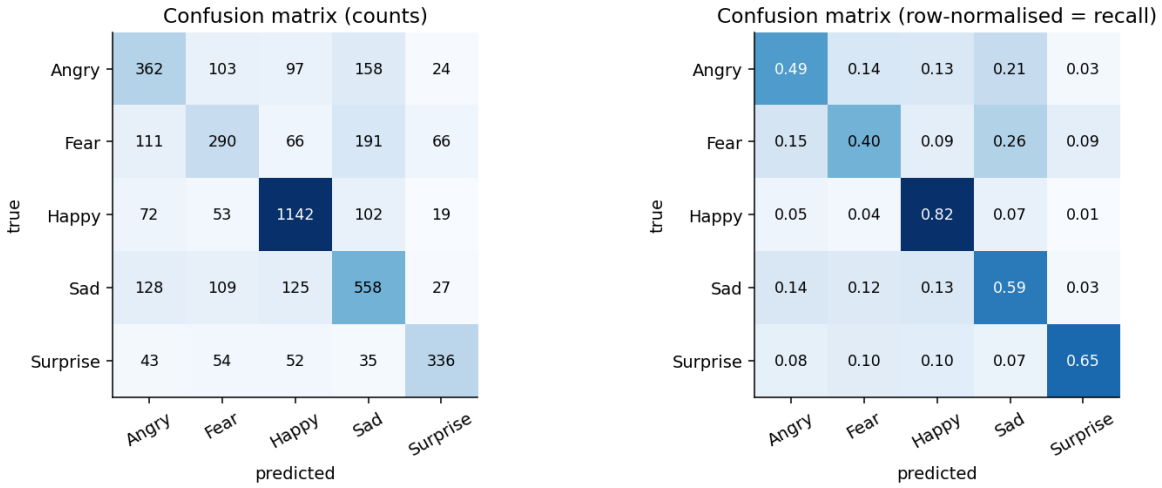


Figure 8. Test confusion matrix of the ensemble (counts, and row-normalised = per-class recall).

Interpretation of the errors. *Happy* is recognised best (F1 0.80, recall 0.82): it has the most training images and a distinctive cue (the smile), which HOG captures well. Surprise (0.68) also has a clear cue (open mouth, raised brows). The three negative emotions are the weak spots: *Fear* (F1 0.44, recall 0.40) is confused with Sad (191 of 724 Fear images, 26 %) and Angry (111); Angry is confused with Sad (158) and Fear (103); Sad is confused with Angry (128), Happy (125) and Fear (109). At 48x48 grayscale these expressions differ by subtle changes of brow and mouth shape, differ between individuals, and the web-collected labels are themselves ambiguous (a face labelled "Sad" can plausibly be read as "Fear"); a perfect score is therefore not realistic. The ensemble improves on single models by about two points of accuracy and macro-F1, which is expected for high-variance, over-fitting models whose errors are partly independent. The predicted confidence is informative (average 0.71 for correct vs 0.53 for wrong predictions), so low-confidence predictions could be rejected in an application.

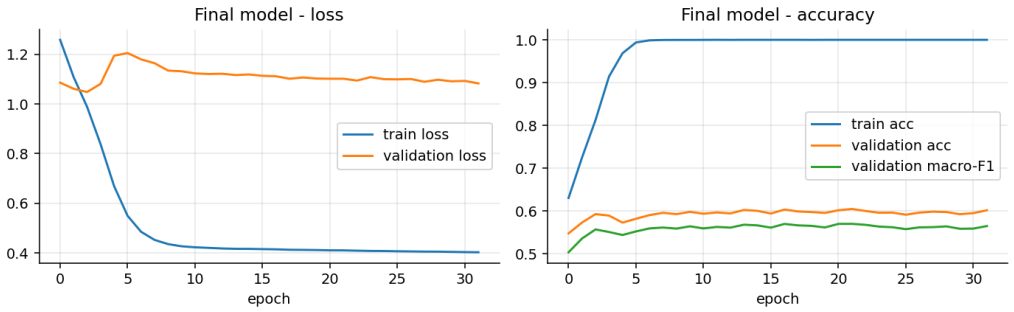


Figure 9. Training curves of one final MLP: training accuracy reaches 100 % within ~6 epochs while validation accuracy stays near 60 % (early stopping selects the best epoch).

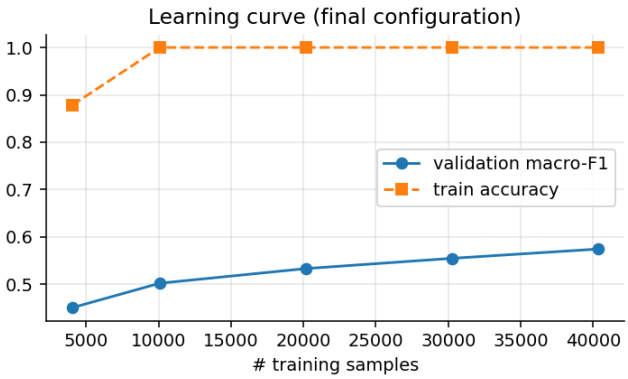


Figure 10. Learning curve of the final configuration (validation macro-F1: 0.451, 0.502, 0.533, 0.555, 0.574 for 10, 25, 50, 75, 100 % of the training data). The curve is still rising, i.e. the model is data-limited. Sample counts include the flipped copies.

5. Discussion

Why do MLPs plateau around 60 %? The training accuracy is ~100 % for almost every configuration while validation accuracy is 50-60 %, so the models are limited by *generalisation* rather than by capacity or optimisation. A fully-connected layer sees an image as an unordered vector of 2,304 numbers: it has no notion that neighbouring pixels are related or that a mouth is the same object when shifted by a pixel, so it must learn every spatial variant from examples. With only ~20k unique training images this is not possible, which explains why (i) features that build in local structure and invariance (HOG) help much more than anything done to the network, (ii) augmentation helps pixel MLPs a lot, (iii) the learning curve is still rising, and (iv) the first-layer weight maps of a pixel MLP (Figure 11) look like noisy, pixel-specific patterns rather than face parts. Convolutional networks, which are excluded by the assignment, build exactly this prior into the architecture.

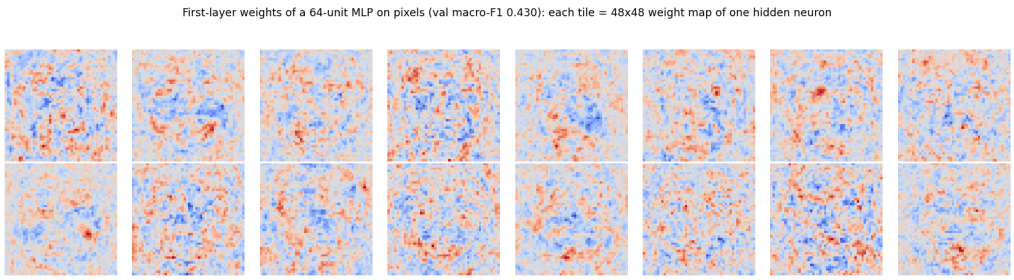


Figure 11. First-layer weights of a small 64-unit MLP on pixels (validation macro-F1 0.430): each tile is the 48x48 weight map of one hidden neuron. No clean eye / mouth detectors are visible.

What actually mattered. (1) Input representation: +10 points. (2) Data cleaning: removing 51 % duplicates does not raise the score but makes it honest. (3) Augmentation and batch size: about +1 point each. (4) Architecture, activation, dropout, weight decay, label smoothing, class weighting: all within about +-1 point, i.e. comparable to the seed-to-seed standard deviation (0.2-0.8 point). Reporting means over several seeds and re-checking the winners with fresh seeds was important: the single-seed scores of the five best trials moved by between -0.7 and +0.4 point when they were re-trained with fresh seeds.

Threats to validity.

- Only *exact* duplicates were removed. Near-duplicates (crops, re-compressions, flipped copies of the same photo) may still link training and test images and make the test score slightly optimistic.
- Labels come from web searches and are noisy, so the achievable accuracy is bounded; no human-accuracy reference is available for this dataset.
- One fixed train/validation/test split and 2-5 seeds per configuration; differences below ~0.5-1 point are not statistically meaningful. The one-factor tuning was greedy and sequential, so interactions were only partly explored (the random search covers some of them).
- The optimiser comparison is confounded by the weight-decay value inherited from the previous stage (Section 3.6), so it only supports conclusions about batch size, AdamW learning rate and the schedule.
- The reported 62.2 % is for an ensemble of five MLPs (still only MLPs); a single deployed model would obtain ~60 %.

6. Conclusion

Using only MLPs, a facial-emotion classifier for five classes reaches **60.1 % test accuracy (macro-F1 0.573) with a single model and 62.2 % (0.593) with a five-model ensemble**, compared with 32 % for the majority class and 47 % for a plain MLP on raw pixels. The gains came mostly from good input features (HOG+LBP), careful data cleaning and moderate tuning of batch size and augmentation; further architecture tuning gave diminishing returns because the models over-fit a small set of unique images. The next steps, outside the MLP constraint, would be convolutional networks, more clean data, and higher-resolution or colour input.

Reproducibility: run emotion_mlp.ipynb top to bottom (the dataset is fetched with `kagglehub`*); fixed seeds; Python 3.11, PyTorch 2.11 (CUDA), scikit-learn 1.7, scikit-image 0.26; about 40 minutes on an RTX 4060 laptop GPU. The notebook also stores all 132 evaluated configurations in results/all_runs.csv and the final metrics in results/final_results.json.*